

# Agent Guild — Task Board (2-Person Execution Plan)

---

## Project Goal

Build a **Generic Agent Builder System** with:

- Agent creation via prompt
  - Tool editing via remarks
  - Orchestrator agent
  - Streamlit UI
- 

## Team Setup

- Person A
- Person B

👉 You will **rotate roles per feature**  
👉 Both will touch **backend + UI**

---

## PHASE 1 — Foundation (Day 1–2)

---

### ◆ Feature 1: Project Setup + Structure

#### Tasks

##### Person A (Backend)

- Create folder structure:

agents/  
core/  
ui/  
orchestrator/  
generator/

- Setup virtual environment
  - Install dependencies (LangChain, Streamlit)
- 

#### Person B (UI)

- Create basic Streamlit app
  - Sidebar placeholder
  - Main layout structure
- 

#### ✓ Output

- App runs
  - Empty UI visible
  - Folder structure ready
- 

## ◆ Feature 2: Agent Loader + Sidebar

---

### Tasks

#### Person A (Backend)

- Implement:

get\_agents()  
load\_config(agent\_name)

---

#### Person B (UI)

- Sidebar:
    - list agents
    - select agent
    - refresh button
- 

## ✓ Output

- Agents auto-detected
  - Selecting agent loads config
- 



## ROTATE ROLES

---



## PHASE 2 — Agent System (Day 3–4)

---

### ◆ Feature 3: Tool System

---

#### Tasks

##### Person B (Backend)

- Define tool interface:

`run(input, llm, config)`

- Create:
    - summarize tool
    - search tool
- 

##### Person A (UI)

- Display tools list
  - Expandable tool panels
- 

### Output

- Tools visible in UI
  - Tool structure finalized
- 

## ◆ Feature 4: Agent Execution

---

### Tasks

#### Person B (Backend)

- Implement:

`create_agent()`

`run_agent(agent_name, query)`

---

#### Person A (UI)

- Chat UI:
    - input box
    - display responses
- 

### Output

- Agent responds to queries
- 



## ROTATE ROLES



## PHASE 3 — Editing System (Day 5–6)

---

### ◆ Feature 5: Tool Editor UI

---

#### Tasks

##### Person A (Backend)

- Load tool config
  - Save updated config
- 

##### Person B (UI)

- Build tool editor:
    - prompt field
    - remarks input
    - apply button
- 

#### ✓ Output

- Tool config editable manually
- 

### ◆ Feature 6: Remarks → Patch System



#### Tasks

##### BOTH (Pair Programming)

- Design LLM prompt
- Implement:

```
generate_patch(tool_config, remark)
apply_patch(config, patch)
```

---

### ✓ Output

- Remarks update tool config automatically
- 



## PHASE 4 — Orchestrator (Day 7–8)

---

### ◆ Feature 7: Orchestrator Agent

---

#### Tasks

##### BOTH (Pair Programming)

- Create orchestrator agent
  - Add tools:
    - create\_agent
    - edit\_tool
    - run\_agent
- 

### ✓ Output

- User can control system via chat
- 



## PHASE 5 — Agent Generator (Day 9)

---

## ◆ Feature 8: Agent Creation

---

### Tasks

#### Person B (Backend)

- Generate:
    - agent folder
    - config.json
    - tool files
- 

#### Person A (UI)

- “Create Agent” UI:
    - prompt input
    - create button
- 

### ✓ Output

- New agent created dynamically
- 



## PHASE 6 — Polish (Day 10)

---

## ◆ Feature 9: UI Improvements

---

### Tasks (Both)

- Add theme (dark mode)
- Improve layout
- Add:
  - preview before apply
  - better spacing

---

---

## ◆ Feature 10: Testing + Fixes

---

### Tasks (Both)

- Test:
    - agent creation
    - editing
    - running
  - Fix bugs
- 
- 

## Daily Workflow

---

### Start of Day

- Pick feature
  - Assign roles
- 

### End of Day

- Merge code
  - Test together
- 

## Rules (IMPORTANT)

---

## Don't:

- Edit same file at same time
  - Skip contracts
  - Mix UI + backend logic
- 

## Do:

- Use Git branches
  - Keep commits small
  - Communicate changes
- 

## Final Outcome

By end of this plan, you will have:

- Fully working agent builder
  - Editable tools via remarks
  - Orchestrator system
  - Clean UI
- 

## Final Insight

You're not just building a project.

You're building:

 **An Agent Operating System**

---

**End of Task Board**